Fingerprint Service

Service berbasis Go untuk Windows yang mengintegrasikan ZKFinger SDK (ZKTeco) via COM/ActiveX. Service mengembalikan data fingerprint dalam bentuk string dan menyediakan pencocokan 1:1 (verify) dan 1:N (identify) menggunakan fitur SDK; data template untuk matching dikirim oleh pemanggil (tidak ada database di dalam service).

Persyaratan

- Windows (driver ZKFinger dan sensor terpasang)
- Go 1.17+
- ZKFinger SDK / driver terinstal (setup dari folder Fingerprint; komponen COM/OCX terdaftar, mis. ZKFPEngX / Biokey.ocx)

Build & Menjalankan Server (Jalankan di Terminal)

go mod tidy
-> Berfungsi untuk mendeteksi, mengunduh, dan merapikan semua *library/package* pihak ketiga.

go build -o bin/server.exe ./cmd/server
-> Proses kompilasi kode Golang menjadi sebuah aplikasi *executable* Windows (`.exe`). Flag `-o bin/server.exe` menginstruksikan Golang untuk menyimpan hasil jadinya ke dalam folder `bin` dengan nama `server.exe`.

bin\server.exe -addr :8080
-> Perintah untuk menyalakan aplikasi server yang baru saja dibuat. Parameter `-addr :8080` mengatur agar REST API Golang ini "mendengarkan" *request* dari Frontend/Laravel di *port* 8080.

Penjelasan Template9 dan Template10

Template9
-> Template9 adalah algoritma ekstraksi sidik jari ZKTeco (versi 9.0) yang sifatnya lebih toleran terhadap kondisi fisik jari. Algoritma ini sangat andal untuk pemakaian sehari-hari karena tetap bisa membaca jari yang basah, kering, atau kotor. Hasil pemindaiannya jarang mengalami *error* atau data kosong, sehingga sangat ideal digunakan agar karyawan tidak kesulitan saat absen rutin.

Template10
-> Template10 adalah algoritma versi 10.0 yang lebih baru, presisi, dan memiliki tingkat keamanan maksimal. Namun, algoritma ini sangat sensitif terhadap kualitas jari. Jika jari sedikit saja basah atau posisinya kurang pas, alat sering gagal membaca dan hanya menghasilkan *string* kosong.

Penjelasan Core Function (Golang Wrapper)

Karena SDK ZKTeco aslinya menggunakan Event-Driven di C# (seperti OnCapture / OnEnroll), kita membuat custom wrapper function di Golang (internal/zkfp/engine_windows.go) agar bisa dipanggil secara sinkron via HTTP REST API. Pendekatan yang kita gunakan adalah Polling + Message Loop.

1. Function Capture()

Function ini bertugas memaksa alat menyala, menunggu jari ditempel, lalu mengekstrak string sidik jarinya

Parameter: timeout (batas waktu tunggu, misal 30 detik)

Return: Format JSON berisi *string* sidik jari versi 9 dan 10.

Alur Kerja (Flow):

`[ CancelCapture() ]` ***(Pembersihan / Reset)***

⬇

`[ BeginCapture() ]` ***(Menyalakan Sensor)***

⬇

`[ GetTemplateAsStringEx(templateFP) ]` ***(Polling / Tunggu Jari Ditempel)***

⬇

`[ CancelCapture() ]` ***(Mematikan Sensor)***

⬇

`[ Return JSON ]` ***(Mengembalikan Template9 & Template10)***

```
{
"Template9": "string_base64_panjang_v9...",
"Template10": "string_base64_panjang_v10..."
}
```

Eksekusi Sukses/Gagal: Jika string sidik jari didapatkan, sensor dimatikan lagi secara paksa via *CancelCapture()* agar tidak terus merekam di latar belakang (Ghost Bug), lalu me-return String Template9 dan Template10. Jika waktu habis, return error timeout.

2. Function Enroll()

Function ini bertugas memaksa alat menyala, menunggu jari ditempel, lalu mengekstrak string sidik jarinya

Parameter: timeout (batas waktu tunggu, misal 30 detik), enrollCount (jumlah jari ditempel, biasanya 3).

Return: Format JSON berisi *string* sidik jari versi 9 dan 10.

Alur Kerja (Flow):

`[ CancelEnroll() ]` *(Pembersihan / Reset)*

⬇

`[ SetEnrollCount(3) ]` *(Set Wajib 3x Tempel)*

⬇

`[ BeginEnroll() ]` *(Menyalakan Sensor)*

⬇

`[ Polling 3x Scan ]` *(Tunggu proses penggabungan 3 jari selesai)*

⬇

`[ CancelEnroll() ]` *(Mematikan Sensor)*

⬇

`[ Return JSON ]` *(Mengembalikan Template Master v9 & v10)*

{
"Template9": "string_base64_panjang_v9...",
"Template10": "string_base64_panjang_v10..."
}

3. Function Verify()

Function untuk mengecek apakah dua string sidik jari berasal dari orang yang sama.

Parameter: regTemplate (String jari dari Database), verTemplate (String jari basah/Live Scan), doLearning (boolean, false).

Return: Format JSON berisi status kecocokan (boolean) dan seberapa tinggi skor kemiripannya.

Alur Kerja (Flow):

`[ Terima Data ]` *(String dari DB & String Jari Live)*

⬇

`[ VerFingerFromStr() ]` *(Fungsi Komparasi Bawaan SDK)*

⬇

`[ Kalkulasi Internal ]` *(Mesin mencocokkan titik minutiae sidik jari)*

⬇

`[ Return JSON ]` *(Mengembalikan boolean Match & nilai Score)*

{
“Match”: true,
“Score”: 85
}

4. Function Identify()

Function untuk mencari 1 sidik jari dari banyak data.

Parameter: templates []TemplateEntry (Array of Struct yang berisi ID dan String Template9 dari database), verTemplate (String jari basah/Live Scan).

Return: Format JSON berisi ID yang cocok (mengembalikan `-1` jika tidak ada yang cocok), skor kemiripan, dan jumlah data yang berhasil diproses.

Alur Kerja:

`[ CreateFPCacheDBEx() ]` *(Membuka Cache Memori di RAM)*

⬇

`[ AddRegTemplateStrToFPCacheDBEx() ]` *(Looping: Masukkan semua jari dari DB ke RAM)*

⬇

`[ IdentificationFromStrInFPCacheDB() ]` *(Pencarian 1 Jari Live vs Ratusan Jari di RAM)*

⬇

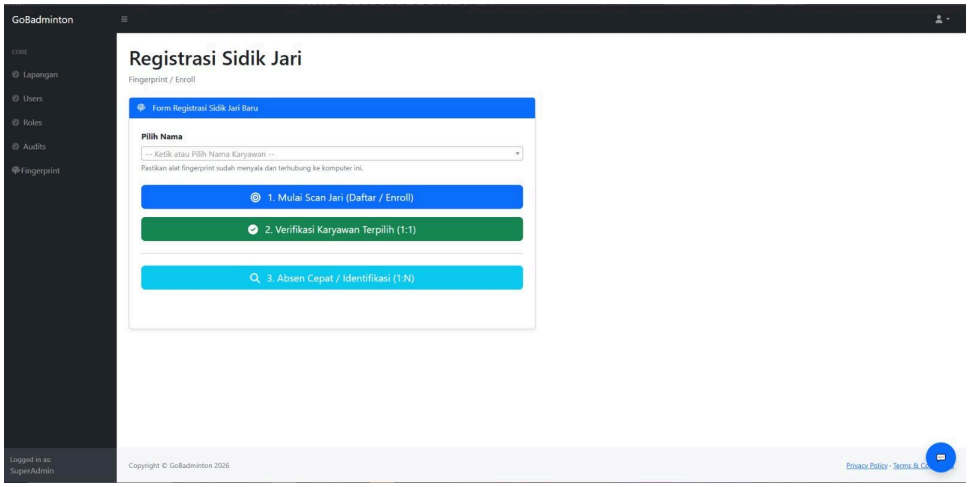
`[ FreeFPCacheDBEx() ]` *(WAJIB: Bersihkan RAM agar tidak Memory Leak)*

⬇

`[ Return JSON ]` *(Mengembalikan MatchedID & Score)*

```
{
"MatchedID": 4,
"Score": 75,
"Processed": 100
}
```

Implementasi Integrasi dengan Laravel & JavaScript



Cara Kerja Flow Pendaftaran Jari (Enroll)

Alur Kerja (Flow):

`[ Pilih Nama Karyawan ]` *(Admin memilih nama di UI Web)*

⬇

`[ POST :8080/enroll ]` *(JS memanggil Golang, user menempelkan jari 3 kali)*

⬇

`[ Return JSON ]` *(Golang mengembalikan string `template9` ke JS)*

⬇

`[ POST /fingerprint/save ]` *(JS mengirim `template9` & `user_id` ke API Laravel)*

⬇

`[ Simpan ke DB ]` *(Controller Laravel menyimpan data permanen ke database)*

Cara Kerja Flow Verifikasi (Verify 1:1)

Alur Kerja (Flow):

`[ Pilih Nama Karyawan ]` *(User/Admin memilih nama di UI Web)*

⬇

`[ GET /fingerprint/data/{id} ]` *(JS mengambil string sidik jari milik user tersebut dari DB Laravel)*

⬇

`[ POST :8080/enroll ]` *(JS memanggil Golang, meminta alat membaca jari aktual user)*

⬇

`[ POST :8080/verify ]` *(JS mengirim kedua string: jari dari DB & jari aktual ke Golang)*

⬇

`[ Return JSON ]` *(Golang memproses dan membalas status `Match` true/false untuk ditampilkan di Web)*

Cara Kerja Flow Identifikasi (Identify 1:N)

Alur Kerja (Flow):

`[ Klik Tombol Absen ]` *(Klien langsung scan tanpa memilih nama di UI Web)*

⬇

`[ GET /fingerprint/data-all ]` *(JS menarik SELURUH data sidik jari karyawan dari DB Laravel)*

⬇

`[ POST :8080/enroll ]` *(JS memanggil Golang, meminta alat membaca jari aktual user)*

⬇

`[ Format Data ]` *(JS menyusun format array data agar sesuai dengan Struct Golang)*

⬇

`[ POST :8080/identify ]` *(JS mengirim array seluruh jari DB & 1 jari aktual ke Golang)*

⬇

`[ Return JSON ]` *(Golang membalas `MatchedID` karyawan yang paling cocok)*

⬇

`[ Tampilkan Hasil ]` *(JS mencocokkan ID tersebut dan memunculkan nama karyawan di Web)*